Uber-like Ride-Hailing Platform — Microservices Build & Learning Plan

Oct 10, 2026 · @Data Engineering 2

Goals

By the end you will own a running ride-hailing platform of 7 services in three Python frameworks, a React frontend, a traffic simulator, and a written log of 25+ production problems you reproduced, diagnosed and fixed. The log is the resume; the app is the proof.

Working name: **RideMesh** (rename as you like).

What "done" means

- Rider and driver web apps: request a ride, see the driver move live on a map, pay, rate.
- Backend: dispatch/matching, pricing with surge, trips, payments, notifications, each its own service and database, talking over REST and Kafka events.
- Runs on Docker Compose on your laptop, then on a Kubernetes cluster (kind or k3d locally, optional cloud).
- Observability: metrics, logs, distributed traces and dashboards for every service.
- A load generator that simulates thousands of drivers and riders so that real problems show up.
- Capstone: migrate the hottest table and then the whole trips database to a new schema and store with zero downtime, under load.

How to learn, not just build (non-negotiable rules)

1. Every feature ships with a load test and a dashboard panel. No numbers, no done.
2. Every production problem gets a 1-page incident report: symptom, how it was detected, root cause, fix, prevention. Keep them in `docs/incidents/`. These become your resume bullets and interview stories.
3. Break things on purpose. Each phase lists problems to inject (kill a pod, double-deliver a Kafka message, slow a database). Fixing injected failures is where the learning is.
4. Design before code: each service gets a half-page design note (responsibilities, owned data, APIs, events, failure modes) before the first line.

Time

Flexible. At 3 to 4 hours a day the plan runs about 16 weeks (≈110 days); full-time it compresses to about 9 weeks. Phase day ranges are in the roadmap below. The order matters more than the dates.

Assumptions and tech stack

Use each framework where its strengths match the service: Django where you need an ORM, admin and transactional business logic, FastAPI where you need async I/O and WebSockets, Flask where a service is small and stateless. Mixing three frameworks in one Python language is realistic (teams choose per need) and gives you an interview story about shared contracts across heterogeneous services.

Assumptions

- Solo developer, laptop with 16 GB RAM. Use Redpanda (Kafka-compatible, single binary) instead of a full Kafka cluster locally; one Postgres instance with one database per service until Kubernetes.
- Web frontend only (rider app, driver app). Mobile is out of scope; the WebSocket and REST contracts would carry over to React Native later.
- No real payment gateway: a mock gateway you write, with configurable latency, timeouts and random failures.
- Maps: MapLibre GL with OpenStreetMap tiles. ETA starts as straight-line distance and later uses OSRM (self-hosted, Docker) for road routing.
- Python 3.12, Postgres 16, Redis 7, Redpanda, Docker, Kubernetes via kind or k3d.

Service to framework mapping

Service	Framework	Why this framework	Owns (data)	Main traffic
Identity	Django + DRF	Auth, driver onboarding, document review and ops admin come almost free with Django admin	users, drivers, vehicles, sessions (Postgres)	Low-rate REST, JWT issuance

Service	Framework	Why this framework	Owns (data)	Main traffic
Trips	Django + DRF	Rich transactional state machine (requested → matched → arriving → in progress → completed/cancelled); Django ORM, migrations and admin suit it. This is the heavy table you migrate in the capstone	trips, trip_events (Postgres; the biggest, hottest table)	REST + Kafka events
Realtime gateway	FastAPI (WebSockets)	Thousands of concurrent long-lived sockets; async is the only sane choice in Python	nothing durable (stateless edge)	Driver GPS pings every 3–5 s in; trip/driver updates pushed out
Location	FastAPI	Async consumer of GPS stream; serves "drivers near point" in milliseconds	driver positions in Redis GEO keyed by H3 cell	High write, high read, no durability needs
Dispatch	FastAPI	Matching loop, async calls to Location and Pricing, Kafka consumer/producer	offers, driver availability locks (Redis + Postgres)	Burst at ride request time; latency-critical
Pricing	Flask	Small, stateless compute: fare estimate, surge multiplier per H3 cell; Flask keeps it tiny	surge snapshots (Redis), tariffs (Postgres)	REST, called by Trips and Dispatch
Payments	Flask + SQLAlchemy + Alembic	Idempotent charge API and a double-entry ledger; SQLAlchemy and Alembic give you a second migration tool to learn beside Django's	ledger entries, charges, payouts (Postgres)	REST + Kafka (trip.completed)

Shared platform

- API gateway: Traefik (simple) or Kong (closer to industry); JWT validation at the edge, rate limits, routing.
- Messaging: Kafka API (Redpanda locally, Strimzi or managed Kafka on Kubernetes). Schemas in JSON Schema or Avro with a registry, versioned.
- Caching and geo index: Redis.
- Observability: OpenTelemetry SDK in every service → Prometheus (metrics), Loki (logs), Tempo or Jaeger (traces), Grafana dashboards.
- Frontend: React 19 + TypeScript + Vite, TanStack Query for server state, Zustand for client state, MapLibre GL for maps, native WebSocket with a reconnect wrapper.
- Tooling: uv, ruff, mypy, pytest, pre-commit, Docker multi-stage builds, GitHub Actions CI, one monorepo with `services/`, `apps/`, `platform/`, `docs/`.
- Load: Locust for HTTP, a custom Python "world simulator" that drives N fake drivers on a city bounding box and M riders requesting trips.

Target architecture

Seven services sit behind two edges (a REST gateway and a WebSocket gateway), each service owns its own database, services call each other synchronously only when a user is waiting for the answer, and everything else flows through Kafka events. This is a small version of Uber's Domain-Oriented Microservice Architecture: domains with a single entry point, layered so lower layers never depend on upper ones, which bounds the blast radius of a failure.

Five layers, top to bottom: Clients (rider web app, driver web app) → Edge (API gateway, Realtime gateway) → Product services (Trips, Dispatch) → Platform services (Identity, Location, Pricing, Payments) → Infrastructure (Kafka, Postgres per service, Redis, observability stack). Calls run downward only; every service publishes and consumes Kafka events while writing only to its own database.

Rules that make it a microservice system rather than a distributed monolith

1. Database per service. No cross-service joins, no shared tables. Another service's data is reached only through its API or its events. Uber found that when teams modify each other's data models, "networked monoliths" form that must be deployed together.
2. Layers only call downward: edge → product (Trips, Dispatch) → platform (Identity, Pricing, Payments, Location) → infrastructure (Postgres, Redis, Kafka). A platform service never calls a product service.

3. Synchronous REST/gRPC only for answers the caller needs now: Trips → Pricing for a fare quote, Dispatch → Location for nearby drivers, anything → Identity to validate a token. Each sync call has a timeout, a retry budget and a circuit breaker.
4. Everything else is an event on Kafka: `trip.requested`, `trip.matched`, `trip.started`, `trip.completed`, `trip.cancelled`, `driver.location`, `driver.availability`, `payment.captured`, `payment.failed`. Each event carries a schema version, an event id and the trip id as partition key so one trip's events stay ordered.
5. Producers write events through a transactional outbox in the same database transaction as the state change; a relay publishes them. Consumers are idempotent and deduplicate on event id. Failed events go to retry topics and then a dead-letter topic, never block the partition.
6. The trip is a state machine: `REQUESTED → MATCHED → DRIVER_ARRIVING → IN_PROGRESS → COMPLETED` with `CANCELLED` reachable from the first four. Every transition is a row in `trip_events` (append-only), which is what the capstone migration later moves.

One ride, end to end

1. Rider web app posts `POST /trips` with an `Idempotency-Key`; the gateway validates the JWT and forwards to Trips.
2. Trips quotes a fare from Pricing (sync, 300 ms timeout, cached surge), writes the trip as `REQUESTED` plus an outbox row, returns 202 with the trip id.
3. Dispatch consumes `trip.requested`, asks Location for drivers in the pickup H3 cell and its ring of neighbours, ranks them by ETA, and sends an offer to the best driver through the Realtime gateway (Redis pub/sub to whichever gateway pod holds that driver's socket). The offer expires in 15 s; on timeout or decline the next candidate is offered.
4. Driver accepts: Dispatch takes a short Redis lock on the driver (so two trips cannot be matched to one driver), publishes `trip.matched`; Trips moves the state; Realtime pushes the driver's position to the rider every 3–5 s from the `driver.location` stream.
5. Driver starts and completes the trip; Trips publishes `trip.completed`; Payments charges the mock gateway with an idempotency key, writes a double-entry ledger (rider debit, driver credit, platform fee), publishes `payment.captured`; Trips marks the trip paid; both apps get a receipt push.

Infrastructure and frontend

- Local: Docker Compose with every service, Postgres, Redis, Redpanda, OTel collector, Prometheus, Grafana, Loki, Tempo. Kubernetes (kind/k3d) from the mid-point of the plan: one namespace, a Deployment and HorizontalPodAutoscaler per service, PodDisruptionBudgets, readiness and liveness probes, Traefik ingress.
- Frontend monorepo packages: `apps/rider-web`, `apps/driver-web`, `packages/api-client` (generated from each service's OpenAPI), `packages/realtime` (WebSocket client with reconnect, backoff and resume), `packages/map` (MapLibre wrappers, marker interpolation). Server state lives in TanStack Query; WebSocket events patch the query cache; Zustand holds only UI state.

Backend production problems to build in on purpose

These 16 problems are the ones Uber and similar companies write incident reports about; each row is something you will reproduce under load, fix with a known pattern, and document. The strongest resume material is the "reproduce, then fix" column: it turns a tutorial project into evidence of production judgment.

#	Problem	Seen in the wild	Reproduce, then fix
1	Retry storms: each hop retries, so a failing service deep in the chain receives R^depth times the traffic and cannot recover	Uber's Nov 2025 outage started in a service 5+ levels deep; retry budgets alone would have added 46–135% load, and error ownership stopped ~9.5M spurious requests (Uber, Sept 2026)	Make Pricing fail 30% of calls while Trips, Gateway and the load tool each retry 3×; watch Pricing collapse. Fix: retry budget (≈10% of traffic), exponential backoff with jitter, retry only idempotent calls and only at the hop nearest the error, circuit breaker, deadline propagation
2	Cascading latency and undebuggable call chains	Uber engineers once had to trace a root cause through ~50 services and 12 teams; a latency spike n levels down cascades upstream (DOMA)	Inject 800 ms latency into Location; ride-request p99 doubles with no error anywhere. Fix: OpenTelemetry traces propagated over REST, Kafka headers and WebSocket frames; per-hop timeouts shorter than the caller's

#	Problem	Seen in the wild	Reproduce, then fix
3	Duplicate delivery: Kafka is at-least-once, so a consumer that crashes after acting but before committing acts twice (double charge, double push)	Uber's Kafka reprocessing design assumes at-least-once and builds retry topics plus a dead-letter topic rather than blocking (Uber)	Kill the Payments consumer mid-batch under load; count double charges. Fix: idempotency keys on the charge API, an inbox table unique on event id, retry topics with delays, DLQ with list/purge/replay tooling
4	Dual-write inconsistency: the DB commit succeeds and the event publish fails (or the reverse), and downstream never hears about the trip	The old Uber fulfillment stack lacked atomicity between writes and had to reconcile mismatches constantly (Uber fulfillment re-architecture)	Stop Redpanda for 20 s during a load run; count trips stuck in REQUESTED. Fix: transactional outbox written in the same transaction, a relay that publishes and marks rows, optionally Debezium CDC
5	One driver matched to two trips: concurrent read-modify-write on the same entity	Uber describes exactly this race (driver going offline while matching links an offer) and last-write-wins overwrites under split brain (same article)	Two riders request within 50 ms of each other next to one driver. Fix: a Redis lock with TTL and fencing token, or `SELECT … FOR UPDATE` plus a version column; state-machine guards that reject illegal transitions

#	Problem	Seen in the wild	Reproduce, then fix
6	Multi-service business transactions (trip complete + driver free + payment) with partial failure	Uber used a Saga coordinator with compensating actions and found debugging multi-entity flows hard; the rewrite added a post-commit task table (LATE) for at-least-once side effects (same article)	Make the mock gateway time out after the trip is COMPLETED. Fix: a saga with explicit compensation (`payment.failed` → trip `PAYMENT_PENDING`, retry schedule), side effects recorded in a task table and executed by workers
7	Hot cells and hot keys: a stadium or airport puts thousands of riders and drivers in one H3 cell, one Redis key, one Kafka partition	Uber buckets marketplace events into H3 hexagons to compute supply and demand and surge per cell (Uber H3)	Simulate a match ending: 2,000 requests in one cell in 60 s. Fix: finer H3 resolution near hotspots, k-ring neighbour search, per-cell rate limits, partition key = cell + salt, surge pricing to shift demand
8	Cache stampede: the surge snapshot expires and every Trips pod recomputes it at once	A standard failure of TTL caches under fan-out	Set surge TTL to 5 s and run 500 rps. Fix: request coalescing (single-flight), stale-while-revalidate, jittered TTLs
9	Database connection exhaustion: pods × workers exceeds Postgres `max_connections`; Postgres uses a process per connection	One of the reasons Uber cited when leaving Postgres in 2016 (InfoQ)	Scale Trips to 10 pods × 16 gunicorn workers against `max_connections = 100`. Fix: PgBouncer in transaction mode, per-pod pool caps, `CONN_MAX_AGE`, async drivers where it matters

#	Problem	Seen in the wild	Reproduce, then fix
10	Slow queries at volume: ORM N+1, sequential scans once `trip_events` passes 50M rows	Universal; every ORM project hits it	Load 50M synthetic trip events; run trip history. Fix: `select_related` / `prefetch_related`, `EXPLAIN ANALYZE`, `pg_stat_statements`, covering indexes, keyset pagination, read replicas
11	Consumer lag and rebalance storms: a deploy triggers a consumer-group rebalance, the location stream backs up, riders see stale drivers	A known Kafka operational hazard	Roll Location during 5,000 drivers pinging. Fix: cooperative sticky assignor, static membership, partitions ⩾ consumers, lag alerts, drop GPS pings older than 10 s
12	Deploys that drop thousands of WebSockets and trigger a reconnect storm	Any long-lived connection service	`kubectl rollout restart` the Realtime gateway with 2,000 sockets. Fix: SIGTERM handling and connection draining, preStop hooks, client reconnect with jittered backoff and resume tokens, readiness gates
13	Timers that die with the node: an offer expiry or a payout timer lives in memory	Uber's old stack needed a special framework (sevnup) to hand off timers when a node died (same article)	Kill the Dispatch pod holding 50 open offers. Fix: durable timers as rows (`fire_at`) polled by workers with `FOR UPDATE SKIP LOCKED`
14	Event schema evolution breaks an old consumer	Microservices at Uber churn with a ~1.5-year half-life, so contracts outlive implementations (DOMA)	Add a required field to `trip.requested`; Payments consumer crashes. Fix: schema registry, backward-compatible changes only, tolerant readers, consumer-driven contract tests

#	Problem	Seen in the wild	Reproduce, then fix
15	No backpressure: GPS pings arrive faster than they are processed; gateway memory grows until OOM	Classic for ingest paths	Push 20 pings/s per driver × 5,000 drivers. Fix: bounded queues, coalesce to latest position per driver, sampling, per-connection rate limits, `429` /close with reason
16	Abuse and noisy neighbours: one client floods ride requests or token refreshes	Edge concern at any public API	Locust script with one rider at 100 rps. Fix: token-bucket limits per user and per IP at the gateway, quotas per service, load shedding by priority

Each problem ends with an incident report in `docs/incidents/NN-<slug>.md` and a Grafana panel that shows the before and after.

Data and database problems

The database is where most ride-hailing outages actually originate, so the plan treats Postgres as a system to be engineered, not a box the ORM talks to. The table `trip_events` is the deliberate pain point: append-only, written on every state change, read by trip history, analytics and the capstone migration.

#	Problem	Seen in the wild	Reproduce, then fix
1	A "harmless" Django or Alembic migration takes an `ACCESS EXCLUSIVE` lock and freezes the trips table under traffic	The commonest self-inflicted outage; `ALTER TABLE` waits for the lock while queuing every query behind it (expand/contract guide)	Run `ALTER TABLE trip_events ADD COLUMN … NOT NULL DEFAULT …` on 50M rows at 300 rps. Fix: `lock_timeout` and `statement_timeout` on every migration, nullable columns first, `CREATE INDEX CONCURRENTLY`, constraints added `NOT VALID` then validated, a CI linter for unsafe migrations

#	Problem	Seen in the wild	Reproduce, then fix
2	Write amplification and index bloat on a hot, heavily indexed table	Uber's stated reason for moving from Postgres to MySQL in 2016: an update to one indexed column rewrites every index and inflates replication traffic (InfoQ)	Add six indexes to `trips` and update `status` 200×/s; measure WAL bytes and replica lag. Fix: fewer, composite indexes; status in a narrow table; HOT updates by keeping updated columns unindexed; partial indexes
3	Replica lag causes read-your-own-write bugs: rider creates a trip, the next GET goes to a replica and returns 404	Standard primary/replica trap	Add a streaming replica with 2 s injected lag and route reads to it. Fix: route reads after a write to the primary for N seconds (session pinning), or compare LSNs; never read trip state for dispatch from a replica
4	Table growth: `trip_events` reaches hundreds of millions of rows; vacuum falls behind, queries degrade, backups take hours	Uber keeps only 12 weeks of hot payment data and tiers the rest to cold storage (Uber ledger migration)	Generate 200M events with the simulator. Fix: declarative range partitioning by month, partition pruning, detaching and archiving old partitions to object storage, autovacuum tuning per table
5	Sharding when one database is not enough: cross-shard queries, hot shards, re-sharding	Uber shards cities into pods and found a vertical scaling ceiling per pod; its new fulfillment store shards by key range and warns about hotspots (Uber fulfillment)	Shard Trips by city into two Postgres databases with a routing layer in the Django DB router. Fix: shard key = city or H3 parent cell, a directory service for shard lookup, scatter-gather for admin queries, consistent hashing to add a third shard

#	Problem	Seen in the wild	Reproduce, then fix
6	Consistency choice: strong vs eventual, and what each costs	Uber's old stack chose availability over consistency and paid in reconciliation; the rewrite chose strong, cross-row transactions (Uber fulfillment)	Make the rider's trip list eventually consistent (read model built from events) while trip state stays strongly consistent. Fix: document which reads tolerate staleness; CQRS read models rebuilt from `trip_events`
7	Idempotent, resumable backfills that do not DDoS your own system	Uber's trillion-record backfill ran in small idempotent batches with rate control (AIMD), an emergency stop, and sampled logging (Uber ledger migration)	Backfill a new `h3_cell` column across 50M rows while load runs. Fix: batches of 10–50k with checkpoints, sleep between batches, adaptive rate from replica lag and p99, kill switch, per-batch stats
8	Data quality: corrupt or half-written rows discovered only during migration	Uber expected corruption at scale and set a correctness target between 99.99% and 99.9999% rather than 100% (Uber ledger migration)	Seed 0.01% bad rows (null pickup, impossible timestamps). Fix: validation jobs that quarantine bad rows instead of halting, a dead-row table, reconciliation reports
9	Audit and money correctness: payments must balance to the cent and never be edited	Uber's LedgerStore is append-only with signed, verifiably immutable records (Uber ledger migration)	Inject a bug that credits the driver twice. Fix: double-entry ledger where every transaction sums to zero, an hourly balance check, append-only tables with revoked UPDATE/DELETE grants

#	Problem	Seen in the wild	Reproduce, then fix
10	Backup and restore that were never tested	Universal	Delete the Payments database on purpose. Fix: `pg_basebackup` + WAL archiving, point-in-time recovery drill with a measured RTO/RPO

Keep two numbers on your dashboard from day one: replica lag and the oldest open transaction; most of the rows above show up there first.

Frontend production problems

The frontend problems that matter in ride-hailing are about real time and bad networks, not component styling. Uber rebuilt its web rider app as a 50 kB core so it could reach interaction in about three seconds on 2G, and loads the map only when measured network speed allows it (Uber m.uber). Your two apps should be judged the same way: by what they do on a slow phone in a moving car.

#	Problem	Reproduce, then fix
1	The driver marker jumps and teleports because GPS pings arrive every 3–5 s with noise	Replay a recorded ping stream. Fix: interpolate between pings with `requestAnimationFrame`, snap to the OSRM route, discard points with impossible speed, bearing-based rotation
2	WebSocket drops on tunnels and tab sleep; the rider stops seeing updates and never knows	Throttle to offline in DevTools for 20 s during a trip. Fix: heartbeat/ping, reconnect with jittered exponential backoff, resume with a last-event id so missed events replay, visible "reconnecting" state
3	Reconnect storm after a deploy: every client reconnects in the same second	Restart the gateway with 1,000 simulated clients. Fix: jitter and server-sent `retry-after`, exponential cap, connection budget on the client

#	Problem	Reproduce, then fix
4	Duplicate or out-of-order events on the client (`trip.matched` arrives after `trip.started`)	Replay events shuffled. Fix: version or sequence numbers on every event, apply only if newer, a small client-side state machine mirroring the server's
5	Optimistic UI gone wrong: "Request ride" shows success, the server rejects (no drivers, payment failed)	Return 409 from Trips. Fix: optimistic updates with rollback in TanStack Query, idempotency key reused on retry so a double tap cannot create two trips
6	Map performance with 2,000 nearby driver markers	Render all drivers in the city. Fix: only request drivers in the viewport's H3 cells, cluster or decimate at low zoom, WebGL layers via MapLibre instead of DOM markers
7	Bundle size and time-to-interactive on slow networks	Lighthouse on throttled 3G. Fix: route-level code splitting, lazy map loading, `source-map-explorer` budget in CI, preconnect to tile and API hosts
8	Token expiry mid-trip: a 401 in the middle of navigation	Set JWT lifetime to 60 s. Fix: refresh-token rotation, a single in-flight refresh shared by all requests, WebSocket re-auth without disconnect
9	Stale UI after background: driver app returns from the background 10 minutes later with old state	Use the Page Visibility API to detect. Fix: on visibility change, resync trip state via REST then resume the socket
10	Debugging user-reported bugs you cannot reproduce	Fix: client-side OpenTelemetry spans and error reporting that carry the same trace id as the backend request, so a rider's complaint maps to a trace

The driver app is the harder one: it is a long-lived session that must survive the network, the OS and your deploys. Treat it as a distributed-systems client, not a form.

Phase-by-phase plan

Eight phases, about 110 project days at 3 to 4 hours a day; each phase ends with a working system, a load test and the incident reports for the problems it introduced. If time is short, cut scope inside a phase (fewer drivers, no OSRM), never a phase: the later ones are where the interview material is.

Day ranges at 3 to 4 hours a day: Phase 0 days 1–7, Phase 1 days 8–20, Phase 2 days 21–38, Phase 3 days 39–50, Phase 4 days 51–64, Phase 5 days 65–78, Phase 6 days 79–92, Phase 7 days 93–110. Full-time work compresses each by roughly 40%.

Phase	Build	Problems you inject and fix (numbers refer to the tables above)	Exit criteria
0 Foundations	Monorepo, uv/ruff/mypy/pytest, pre-commit, Docker Compose with Postgres, Redis, Redpanda, OTel collector, Prometheus, Grafana, Loki, Tempo; Traefik; a hello service in each framework; design-note and incident-report templates; event catalog v1	None injected. Write the first ADRs: why three frameworks, why outbox, why H3	A request through Traefik to each framework shows as one trace in Grafana
1 Identity + Trips	Django Identity (JWT, refresh rotation, driver onboarding, admin); Django Trips with the state machine, `trip_events`, OpenAPI, idempotency keys, transactional outbox + relay; rider web login and ride request (list view, no map); Locust at 100 rps	Backend 3 (basic dedupe), 4 (outbox), 5 (state guards); data 1 (first migration with `lock_timeout`)	100 rps ride requests, zero duplicate trips under a double-tap test, trips visible in admin

Phase	Build	Problems you inject and fix (numbers refer to the tables above)	Exit criteria
2 Realtime + Dispatch	FastAPI Realtime gateway (WebSockets, Redis pub/sub fan-out); Location (H3 index in Redis GEO, k-ring queries); Dispatch (offer loop, driver lock, durable timers); world simulator with 1,000 drivers; driver web app with MapLibre; rider sees driver move	Backend 5 (double match), 7 (hot cell), 11 (lag), 13 (timers), 15 (backpressure); frontend 1, 2, 4	1,000 simulated drivers, p95 match under 8 s, no driver double-matched across 10,000 trips
3 Pricing + Payments	Flask Pricing (fare, surge per cell, cached); Flask Payments (SQLAlchemy, Alembic, mock gateway with failures, double-entry ledger, idempotent charge API); saga for trip completion; receipts in both apps	Backend 3 (double charge), 6 (saga), 8 (stampede); data 9 (ledger balance)	Ledger balances to zero after 10,000 trips with 10% gateway failures
4 Resilience + SLOs	Retry budgets, backoff with jitter, circuit breakers, deadline propagation, gateway rate limits, graceful shutdown, DLQ list/purge/replay CLI, SLOs with alerts, chaos scripts (`make chaos-<name>`)	Backend 1 (retry storm), 2 (latency cascade), 12 (socket drops), 14 (schema break), 16 (abuse); frontend 3, 5, 8	Pricing at 30% errors no longer takes down ride requests; alerts fire before users notice
5 Kubernetes + scale	kind cluster, Kustomize or Helm, HPA, PDB, probes, PgBouncer, a read replica, GitHub Actions CI/CD with canary rollout, secrets	Backend 9 (connections), 10 (slow queries); data 3 (replica lag); frontend 6, 7	Rolling deploy of every service under load with zero failed requests

Phase	Build	Problems you inject and fix (numbers refer to the tables above)	Exit criteria
6 Data at scale	Generate 200M `trip_events`; monthly partitioning and archival; migration linter in CI; backfill framework (batches, AIMD rate, kill switch); shard Trips by city across two databases	Data 2, 4, 5, 7, 8, 10	Trip history p99 under 100 ms at 200M rows; a tested restore
7 Capstone migration	Zero-downtime migration of `trip_events` to a new schema and a new store (section below), then packaging: README with architecture, incident index, demo video, resume bullets	Everything in the capstone section	Migration completes with load running and zero failed requests; old table dropped only after a week of fallback silence

Capstone: zero-downtime migration of a heavy, live data system

The capstone moves `trip_events` (200M rows, written on every state change) to a new schema and then to a new store while the simulator keeps 1,000 drivers and 300 rps of ride requests running, with the rule that no request fails and no event is lost. It is a two-part exercise because real migrations come in both shapes: a schema change inside one database, and a move between stores. The playbook is the one Uber used to move more than a trillion ledger entries over two years with no downtime: dual writes, shadow validation, idempotent incremental backfill, fallback reads, and a long tail before dropping the old store (Uber ledger migration).

Part A — schema migration in place (expand, migrate, contract)

Goal: split the JSON `payload` column into typed columns, add `h3_cell`, and change the primary key from a serial integer to a UUIDv7, with the table partitioned by month.

1. Expand: add new nullable columns and the new partitioned table; every `ALTER` runs with `lock_timeout = '3s'` and `statement_timeout`; indexes are built `CONCURRENTLY`; constraints added `NOT VALID` and validated later. Deploy code that

reads old columns and writes both. Never expand and contract in the same release (expand/contract guide).

2. Backfill: a resumable job copies historical rows in batches of 10–50k ordered by primary key, sleeps between batches, and adapts its rate to replica lag and write p99 (additive increase, multiplicative decrease). Each batch is idempotent (`ON CONFLICT DO NOTHING`), writes a checkpoint and its stats, and the job has a kill switch and sampled logging, as Uber's backfill guidance describes.
3. Validate: a shadow reader serves every production read from both old and new paths and compares; target at least 99.99% agreement, and investigate anything above a stated upper bound rather than chasing every mismatch. An offline job compares full row counts and checksums per partition, and its diff feeds an incremental re-backfill.
4. Switch reads behind a feature flag by percentage (1%, 10%, 50%, 100%) with fallback to the old path on miss; log every fallback and check each one.
5. Contract: after a week with zero fallbacks, stop writing the old columns, revoke writes, keep the old data a further month, take a final backup, then drop.

Part B — moving the store

Goal: move `trip_events` out of the Trips Postgres into a separate event store (a second Postgres cluster partitioned by month, or ClickHouse if you want an analytics angle), with Trips reading recent events locally and history from the new store.

1. Change data capture: Debezium streams `trip_events` inserts from the Postgres WAL into Kafka; a writer service applies them to the new store idempotently (event id as key). This gives you dual writes without touching application code.
2. Backfill history through the same writer from a snapshot, with the same rate control, while CDC keeps the tail current; measure and alarm on replication lag.
3. Shadow validation on reads, percentage cutover with fallback, then stop the CDC and archive the old partitions, exactly as in Part A.
4. Rollback plan per step, written before each step starts: for the cutover it is "flip the flag back"; for the contract step it is "restore from the retained copy", and you rehearse it once.

What to measure during the whole capstone (one Grafana board)

- Failed requests and p99 for ride request, trip history and driver updates, before, during and after.
- Backfill rate, batches completed, rows skipped to the quarantine table, estimated time remaining.

- Replica lag, oldest transaction, lock waits, WAL generation rate, disk growth.
- Shadow-validation agreement rate and fallback count.

What you can then say in an interview: that you migrated a 200M-row live table and then its store with zero failed requests under 300 rps, using expand/contract, CDC-driven dual writes, adaptive backfill, and shadow validation with a stated correctness bound, and that you rehearsed the rollback. Few candidates can say this; it is the single most valuable item on the plan.

Coding principles and engineering practices

The same layout, contracts and tests in every service, whatever the framework, is what makes seven services maintainable by one person and readable by an interviewer in ten minutes.

Inside a service (clean architecture, kept small)

- Four layers per service: `domain/` (entities, state machine, pure Python, no framework imports), `application/` (use cases, ports as Protocols), `adapters/` (ORM repositories, Kafka producers, HTTP clients), `api/` (Django views, Flask blueprints or FastAPI routers, thin). Dependencies point inward only.
- Business rules live in the domain, not in views or models: `Trip.accept(driver_id)` raises `IllegalTransition`, and that is tested without a database.
- Every outbound call goes through a port with a timeout, retry policy and circuit breaker configured in one place; no bare `requests.get` anywhere.
- Configuration from environment with a typed settings object (pydantic-settings); secrets never in the image.
- Structured JSON logs with trace id, request id, user id; no print statements.

Contracts between services

- OpenAPI for every REST API, generated from code and checked into `contracts/`; the frontend client is generated from it in CI.
- Every Kafka event has a JSON Schema in `contracts/events/`, a version field, and a producer test that validates against it. Breaking changes are new event names, not edits.
- Consumer-driven contract tests: a consumer records the fields it reads; the producer's CI fails if it removes one.

Testing pyramid per service

- Unit tests for the domain (fast, most numerous), integration tests with real Postgres and Redpanda via testcontainers, and one end-to-end scenario per phase run in CI against Docker Compose (request a ride, get matched, complete, pay).
- Load tests in `load/` are code, versioned, and run on a schedule; results are saved so regressions are visible.
- Chaos scripts in `chaos/` are first-class: `make chaos-kill-dispatch`, `make chaos-slow-location`.

Engineering hygiene

- Trunk-based development with short branches; CI runs lint, types, tests, migration safety linter, image build and a compose smoke test on every pull request to yourself.
- One ADR (architecture decision record) per significant choice in `docs/adr/`; 15 to 25 by the end. Interviewers love ADRs because they show the alternatives you rejected.
- Semantic commits, a CHANGELOG per service, and a README per service with its design note, APIs, events, dashboards and runbook.
- Security basics: JWT with short lifetimes and rotation, Argon2 password hashing, rate limits, input validation at the edge, least-privilege database roles (the app role cannot `DROP`), dependency scanning in CI.

Interview prep and resume bullets

Write each resume bullet only after the incident report exists, and keep the numbers from your own load tests; a bullet with a measured before and after beats any adjective. Replace the placeholder numbers below with yours.

Resume bullets (template, fill with measured numbers)

- Designed and built a ride-hailing platform of 7 microservices (Django, Flask, FastAPI) with event-driven integration over Kafka, per-service Postgres, Redis geo-index (H3), and React/TypeScript rider and driver apps with live maps; ran it on Kubernetes with full OpenTelemetry tracing.
- Eliminated retry-storm amplification: a 30% failure rate in the pricing service previously multiplied load 8× on downstream services; retry budgets, circuit breakers and deadline propagation held it to 1.1× with ride-request success above 99.9%.
- Guaranteed exactly-once effects over at-least-once Kafka delivery using a transactional outbox, idempotency keys and an inbox table; zero duplicate charges across 100k simulated trips with injected consumer crashes.

- Prevented driver double-matching under concurrent requests with fenced Redis locks and state-machine guards; 0 conflicts in 10k trips at 1,000 concurrent drivers.
- Scaled `trip_events` to 200M rows with monthly partitioning, keyset pagination and index redesign; trip-history p99 from 2.1 s to 80 ms.
- Migrated a 200M-row live table and then its store with zero failed requests under 300 rps using expand/contract, CDC dual writes, adaptive backfill (AIMD, kill switch) and shadow validation with a 99.99% agreement gate; rehearsed rollback.
- Made real-time delivery resilient: WebSocket gateway with graceful draining, resumable streams and jittered reconnects; deploys with 2,000 open connections caused zero lost trip updates.

Questions you must be able to answer from this project

1. Why three frameworks? What did each cost you? (Answer with the mapping table and one pain from each.)
2. Walk me through a ride request end to end, then tell me what happens if Kafka is down for a minute. (Outbox, 202 response, relay catch-up.)
3. How do you make a payment exactly once? (Idempotency key at the API, inbox dedupe at the consumer, ledger invariants.)
4. How did you find the latency problem? (Trace from the gateway to Location; the panel that showed it.)
5. Why H3 and not a plain radius query? (Cells as buckets for supply, demand and surge; k-ring neighbours; hotspot handling.)
6. What is the blast radius of Pricing going down? Of Identity? (Layering; cached surge as a fallback; token validation at the edge.)
7. Explain expand/contract. What locks did you avoid and how did you know? (`lock_timeout`, `pg_locks`, the migration linter.)
8. How did you validate the migration was correct? (Shadow reads, offline checksums, the agreement bound and why not 100%.)
9. What would you do differently with a team of five? (Service boundaries by team, a domain gateway, fewer frameworks.)
10. What breaks first at 10× load? (Give your bottleneck ranking from the load tests.)

Study alongside (one hour a week): Designing Data-Intensive Applications (chapters 5, 7, 9, 11), the Uber engineering blog posts in the sources, and the Postgres documentation on locking and partitioning.

Deploying it for free

The whole system can run at $0 on a permanent basis: a single Arm VM from Oracle Cloud's Always Free tier running k3s (a real Kubernetes), Grafana Cloud's free tier for observability, Cloudflare Pages for the two frontends, GitHub Actions and GHCR for CI and images, and self-hosted Postgres, Redis and Redpanda on the VM. That is closer to a real production setup than any PaaS button, because you operate the cluster, the database and the deploy pipeline yourself. For the last six weeks you can add a time-boxed managed Kubernetes cluster on trial credits to see the fully managed version of the same thing.

What is actually free right now (checked 10 October 2026)

Piece	Free option	Limits that matter	How you use it
Cluster host	Oracle Cloud Always Free: Arm `VM.Standard.A1.Flex`, 2 OCPUs and 12 GB RAM total (reduced from 4/24 in 2026), 200 GB block storage, 1 flexible load balancer (10 Mbps), 10 TB/month egress, plus two 1 GB AMD micro VMs	Instances idle for 7 days (CPU, network and memory all under 20%) can be reclaimed; A1 capacity is sometimes unavailable in a region; a credit card is required but not charged unless you upgrade	One A1 VM with the full 2 OCPU/12 GB runs k3s and everything stateful; the two micro VMs can be k3s agents or a bastion
Kubernetes	k3s on that VM (single node, Traefik and local-path storage built in)	One node means no real multi-node scheduling; still gives Deployments, HPA, probes, rolling updates, Ingress, cert-manager	Everything in Phase 5 works unchanged; for node-failure experiments use kind on your laptop

Piece	Free option	Limits that matter	How you use it
Managed Kubernetes (optional, time-boxed)	Trial credits from Google Cloud, Azure, AWS, DigitalOcean or Civo (typically $100–300 for 30–90 days; check current terms when you start)	Credits expire; set a budget alert and delete the project at the end	Run Phases 5–7 once on a managed cluster with cloud load balancers and autoscaling, then move back to the free VM
Postgres	Self-hosted on the VM (CloudNativePG operator or a StatefulSet on a 150 GB block volume) for Trips and Payments; Neon Free for the small databases	Neon: 100 projects, 100 CU-hours per project per month, 1 GB per project and 20 GB total, compute suspends after 5 min idle (adds cold-start latency, a real-world gotcha)	One Neon project per small service (Identity, Pricing) gives a managed-database experience; the heavy `trip_events` table stays on the VM where you have 150 GB
Kafka	Redpanda single node on the VM, memory capped at 1 GB	No permanently free managed Kafka with real throughput exists; Confluent Cloud and Redpanda Cloud give short-lived sign-up credits only	Self-host for the whole project; spend one week of Confluent credits in Phase 5 to see a managed Schema Registry and consumer-lag tooling
Redis	Self-hosted on the VM (128–256 MB)	Managed free tiers meter commands per day; 1,000 drivers pinging every 4 s is ~21M commands a day, far beyond them	Self-host; note the metering problem in an ADR

Piece	Free option	Limits that matter	How you use it
Observability	Grafana Cloud Free: 10,000 active metric series, 50 GB each of logs, traces and profiles per month, 14-day retention, 3 users, 500 virtual-user hours of k6 load testing	A small cluster with node-exporter and kube-state-metrics can exceed 10k series within days: scrape every 60 s and drop pod-level labels	Grafana Alloy (one agent) on the VM ships metrics, logs and traces; no Prometheus, Loki or Tempo to host; k6 Cloud runs your load tests from outside the VM
Frontends	Cloudflare Pages, Vercel or Netlify free tiers (static React builds)	Hobby tiers are for personal projects; fine here	Build in GitHub Actions, deploy on push; API base URL and WebSocket URL from environment
CI/CD and images	GitHub Actions (free for public repositories) and GHCR for images; Argo CD on the cluster for GitOps	Private repos get a monthly minute allowance; keep the repo public, it is your portfolio anyway	Build multi-arch images (`linux/arm64` for the VM, `linux/amd64` for your laptop) with buildx
DNS and TLS	A free wildcard DNS such as nip.io or sslip.io pointing at the VM IP, or Cloudflare free DNS if you own a domain; cert-manager with Let's Encrypt	Let's Encrypt rate limits per domain per week	Real HTTPS and WSS endpoints, which the WebSocket work needs
Backups and secrets	OCI Object Storage (20 GB free) for WAL-G or pgBackRest backups; OCI Vault (150 free secrets) or SOPS with age	20 GB is enough for compressed base backups plus WAL for the small databases, not for a 200M-row table	Phase 6's restore drill uses this

Fitting it in 12 GB

Component	Memory budget	Notes
k3s and system	1.5 GB	Includes Traefik, CoreDNS, cert-manager, Argo CD
Postgres	2 GB	`shared_buffers` 512 MB; 150 GB block volume
Redpanda	1 GB	`--memory 1G --smp 1`, single broker
Redis	0.25 GB	`maxmemory 200mb`, allkeys-lru only for caches
7 services	2.5 GB	1 replica each at 200–300 MB; 2 replicas for Trips and Realtime so rolling deploys are real
Grafana Alloy and OTel collector	0.5 GB	Sampling on traces at 10% under load
Headroom	4 GB	Page cache for Postgres, spikes during deploys

The real constraint is 2 OCPUs: expect 50–100 ride requests per second on the VM across seven Python services, not the 300 rps of your laptop runs. Run the simulator and Locust from your laptop or k6 Cloud, never on the VM, so load generation does not compete with the system under test.

Setup steps (Phase 5, about 3 days)

1. Create the Oracle account, pick a home region that shows A1 capacity, and launch one `VM.Standard.A1.Flex` instance with 2 OCPUs, 12 GB, Ubuntu 24.04 (arm64), a 50 GB boot volume and an attached 150 GB block volume. Open only ports 22, 80, 443 in the security list and restrict 6443 to your own IP.
2. Install k3s (`curl -sfL https://get.k3s.io | sh -`), copy the kubeconfig to your laptop, and install cert-manager and Argo CD with Helm.
3. In GitHub Actions, build each service with `docker buildx --platform linux/arm64,linux/amd64`, push to GHCR, and update the image tag in `deploy/` with a commit; Argo CD syncs the cluster from that directory. Your first deploy to the cluster is a git push, which is how it works at most companies.

4. Deploy Postgres (CloudNativePG, one cluster, storage on the block volume), Redis and Redpanda with resource limits from the table above; run the Django and Alembic migrations as Kubernetes Jobs with `lock_timeout` set.
5. Install Grafana Alloy with your Grafana Cloud credentials; set the scrape interval to 60 s; import your dashboards from `platform/grafana/` as code.
6. Point `*.ridemesh.<ip>.nip.io` at the VM, create Ingress resources for the gateway and the WebSocket endpoint with cert-manager annotations, and verify `wss://` works from the driver app.
7. Deploy the two frontends to Cloudflare Pages with the API and WebSocket URLs as build environment variables; set CORS on Traefik.
8. Keep the simulator running a few drivers around the clock, which also keeps the VM above Oracle's idle-reclamation thresholds, and run the full load tests from your laptop or k6 Cloud.
9. For the capstone, run Part A on the VM with a 50M-row `trip_events` (the block volume holds it) and the full 200M-row run locally in Docker; the procedure and the numbers you report are the same.

Extra, if you qualify: the GitHub Student Developer Pack adds cloud credits and a free domain for a year, which makes the managed-cluster weeks and real DNS cost nothing too.

Sources

Pages read for this plan, as of 10 October 2026. Read the Uber ones in full before the phase that uses them.

Source	What it gives the plan
Introducing Domain-Oriented Microservice Architecture (Uber, 2020)	Why microservices, their complexity cost at ~2,200 services, layering by blast radius, gateways, the 1.5-year service half-life
How Uber Protects Against Retry Storms (Uber, Sept 2026)	Retry amplification math, retry budgets, error ownership, the Nov 2025 outage numbers

Source	What it gives the plan
Uber's Fulfillment Platform: Ground-up Re-architecture (Uber, 2021)	Trip and supply entities, concurrent read-modify-write races, saga and post-commit task tables, statecharts, consistency vs availability, migration with shadow testing
Migrating a Trillion Entries of Uber's Ledger Data from DynamoDB to LedgerStore (Uber, 2024)	The zero-downtime migration playbook: shadow and offline validation, correctness bounds, incremental idempotent backfill, rate control, emergency stop, fallback reads
Building Reliable Reprocessing and Dead Letter Queues with Apache Kafka (Uber, 2018)	Retry topics, dead-letter topics, non-blocking reprocessing
H3: Uber's Hexagonal Hierarchical Spatial Index (Uber)	Cells as buckets for supply, demand and surge; neighbour rings; resolution hierarchy
Building m.uber (Uber, 2017)	Lightweight web client, 50 kB core, conditional map loading on slow networks
Uber Engineering Moving from Postgres to MySQL (InfoQ, 2016)	Write amplification, replication traffic, process-per-connection; read with the Postgres community rebuttals
Zero-Downtime Database Schema Migration Strategies (Bytebase)	Expand/contract steps, lock levels, online schema change tools, pause thresholds
Zero-downtime migrations: patterns that actually work (47Network)	`lock_timeout`, `CONCURRENTLY`, nullable-first, batched backfills with sleep